System Design & High-Level Architecture (HLD) Handbook

Comprehensive blueprint covering horizontal scaling, load balancers, caching, database sharding, CAP theorem, and concrete production blueprints for TinyURL, Rate Limiters, and Notification Pipelines.

LITHAVEN CAMPUS STUDY VAULT • OFFICIAL REVISION CAPSULE
COURSE: Distributed Systems & System Design | **TARGET:** SDE-2 / Senior Engineering & Architecture Interviews

1. Core Distributed Systems Tradeoffs & Guarantees

Theorem / Concept	Core Principle	Practical Industry Trade-off	Real-World Examples
CAP Theorem	In network partition, choose Consistency (CP) or Availability (AP).	CP sacrifices latency/uptime; AP allows stale reads across nodes.	CP: HBase, Spanner, Zookeeper; AP: DynamoDB, Cassandra
PACELC Theorem	If Partition (P): A vs C. Else (E): Latency (L) vs Consistency (C).	Evaluates database latency trade-offs during normal fault-free operation.	MongoDB: PC/EC; Cassandra: PA/EL
Consistent Hashing	Distributes keys across hash ring using virtual nodes to prevent skew.	Adding/removing nodes only re-keys K/N entries instead of all K.	DynamoDB, Memcached, Akamai CDN Edge
Caching Strategies	Write-Through, Write-Back (Write-Behind), and Cache-Aside (Lazy).	Write-Back has highest throughput but risks data loss on node failure.	Redis, Memcached, CDN Varnish caching

2. Benchmark System Design Blueprints

Design Target	Traffic Profile	Core Architecture & Data Stores	Key Bottleneck & Solution
URL Shortener (TinyURL)	Read-heavy (100:1 read:write). 500M URLs/mo.	Base62 encoding (62^7 = 3.5T URLs), Key Generation Service (KGS), Redis Cache, NoSQL/PostgreSQL.	Hash collisions: Use pre-generated non-colliding KGS IDs stored in DB tokens.
Distributed Rate Limiter	Low-latency (<2ms), millisecond accuracy across cluster.	Redis Cluster with Lua Scripting using Sliding Window Log or Token Bucket algorithm.	Race conditions: Execute atomic Redis Lua script (evalsha) to update counts and timestamps.
Notification Pipeline	High volume (billions/day), bursty, multiple channels.	RabbitMQ / Kafka topics partitioned by user ID, priority worker pools, APNS / FCM / Twilio third-parties.	Downstream vendor rate limits: Implement exponential backoff dead-letter queues (DLQ).
Key-Value Store (Dynamo)	High write throughput, high availability across regions.	Consistent hashing ring, vector clocks for conflict resolution, sloppy quorum (W+R > N).	Network split-brain: Gossip protocol for node membership and Merkle trees for anti-entropy.

3. Back-of-the-Envelope Mental Constants & Estimation Rules

- **Power of Two Rules:** 2^10 = 1 Thousand (1 KB) | 2^20 = 1 Million (1 MB) | 2^30 = 1 Billion (1 GB) | 2^40 = 1 Trillion (1 TB).
- **Daily Traffic Conversion:** 1 Million requests/day = 12 QPS average, ~25-30 QPS peak. 100 Million req/day = ~1,200 QPS avg, ~3,000 QPS peak.
- **Latency Numbers Every Engineer Must Know:** L1 cache: 0.5ns | Mutex lock/unlock: 25ns | Main RAM read: 100ns | SSD random read: 150us | Cross-datacenter RTT (US to EU): 150ms.
- **80/20 Caching Rule:** Cache 20% of the most active read data in RAM. For 500GB daily working set, size Redis cache at 100GB to serve 80%+ hits directly.
- **Database Storage Sizing:** Always multiply raw row payload by 1.4x to account for B-tree index overhead, write-ahead logs (WAL), and storage fragmentation.